Information Modeling Using EXPRESS — Advanced Concepts

Extensibles

- SELECT specifies a list of things to choose from
- ENUMERATION specifies a list of items (names)
- It is possible to extend these lists

```
TYPE approval = EXTENSIBLE ENUMERATION OF
     (approved, rejected); END_TYPE;

TYPE your_approval = ENUMERATION BASED_ON
     approval WITH (cancelled); END_TYPE;

TYPE my_approval = EXTENSIBLE ENUMERATION
     BASED_ON approval WITH (pending);
END_TYPE;
```

Relationships

When an attribute of an entity is of type entity, then a relationship is established between the two entity types.

```
ENTITY door;
  hinges : SET [2:?] OF hinge;
  knob   : handle;
  ...
END_ENTITY;

ENTITY hinge;
  id : name;
  ...
END_ENTITY;

ENTITY handle;
 ...
END_ENTITY;
```

Cardinality

- Cardinality constraint from the referring entity to the referenced entity is specified via the attribute definition (e.g OPTIONAL, aggregation).
- Default cardinality constraint from the referenced to the referring entity is zero to many `[0:?]`.
- `INVERSE` attributes can be used to specify other cardinalities.

Cardinality (cont)

```
ENTITY door;
  hinges : SET [2:?] OF hinge;
  knob   : handle;
  ...
END_ENTITY;

ENTITY handle;
 ...
INVERSE
  knob_for : SET [0:1] OF door FOR knob;
END_ENTITY;
```

Here, the INVERSE clause states that a handle is ‘used’ by zero or one doors.

ABSTRACT ENTITY

An ENTITY that can only be instantiated via its SUBTYPEs.

```
ENTITY general_approval ABSTRACT;
  approved_items : SET OF GENERIC_ENTITY;
  status         : aproval_status;
END_ENTITY;
```

- GENERIC_ENTITY stands for any ENTITY
- All attributes must have specific types before instantiation is possible.
- An attribute type can be redeclared to a more specific kind in a SUBTYPE

Example person ENTITY

```
ENTITY person;
  first_name : STRING;
  last_name  : STRING;
  nickname   : OPTIONAL STRING;
```

```
  ss_no      : INTEGER;
  gender     : sex;
  spouse     : OPTIONAL person;
  children   : SET [0:?] OF person;
UNIQUE
  un1 : ss_no;
WHERE
  w1 : (EXISTS(spouse) AND
        gender <> spouse.gender)
       OR NOT EXISTS(spouse);
END_ENTITY;
```

IS-A Relationship

- EXPRESS supports the IS-A relationship via subtyping.
- Entities S1, S2, ... can be declared to be SUBTYPES of entity E. This also effectively declares E to be a SUPERTYPE of S1, S2, etc.

  That is, S1 is-a E, S2 is-a E, etc. Also, E may-be an S1, E may-be an S2.
- An entity may be both a SUB- and a SUPERTYPE.
- An entity may be a SUBTYPE of more than one entity.
- SUPER/SUBTYPING may be used for many purposes.

Inheritance

A SUBTYPE inherits all the attributes and constraints of its SUPERTYPE(s).

```
ENTITY person;
  first_name : STRING;
  last_name  : STRING;
  ss_no      : INTEGER;
  children   : SET [0:?] OF person;
UNIQUE
  un1 : ss_no;
END_ENTITY;

ENTITY male
  SUBTYPE OF (person);
  wife : OPTIONAL female;
END_ENTITY;

ENTITY female
  SUBTYPE OF (person);
  husband : OPTIONAL male;
END_ENTITY;
```

SUBTYPE instance constraints

- In general, an instance of a Supertype may involve instances of zero or more of its Subtypes.

```
ENTITY person;
  ...
END_ENTITY;

ENTITY employee
  SUBTYPE OF person;
  ...
END_ENTITY;

ENTITY student
  SUBTYPE OF person;
  ...
END_ENTITY;
```

- If this is not the required behaviour, then the 'instance set' can be constrained.

SUBTYPE_CONSTRAINT

```
SUBTYPE_CONSTRAINT sc FOR ent;
-- constraints
END_SUBTYPE_CONSTRAINT;
```

specifies SUBTYPE constraints for ENTITY ent.

Several SUBTYPE_CONSTRAINTs can be specified for any given ENTITY. The constraints are additive.

SUBTYPE Constraint Summary

- No constraints: An instance of the Supertype involves zero or more Subtype instances.
- ABSTRACT SUPERTYPE: An instance of the Supertype must involve one or more Subtype instances.
- TOTAL_OVER(x,y) means that every instance of the Supertype must involve an instance of at least one of the listed Subtypes.
- ONEOF(x,y,z) means that one and only *one of* the listed Subtypes can be instanced with an instance of the Supertype.
- (x ANDOR y) means that an instance of the Supertype may be accompanied by instances of the Subtypes x *and/or* y (the default condition).
- (x AND y) means that an instance of the Supertype may be accompanied by instances of the

Subtypes x *and* y.

ABSTRACT SUPERTYPE

- An entity does not have to declare itself to be a SUPERTYPE. It is a SUPERTYPE if it is mentioned by a SUBTYPE.
- In some cases, a Supertype is not to be instantiated without one of its Subtypes. The entity can be constrained to be an ABSTRACT SUPERTYPE.

```
ENTITY mammal
  ...
END_ENTITY;

SUBTYPE_CONSTRAINT sc_abs FOR mammal;
  ABSTRACT SUPERTYPE;
END_SUBTYPE_CONSTRAINT;

ENTITY dog
  SUBTYPE OF mammal;
  ...
END_ENTITY;
```

TOTAL_OVER

```
ENTITY person;
...
END_ENTITY;

SUBTYPE_CONSTRAINT adultchild FOR person;
  TOTAL_OVER(adult,child);
END_SUBTYPE_CONSTRAINT;

ENTITY child SUBTYPE OF (person);
END_ENTITY;

ENTITY adult SUBTYPE OF (person);
END_ENTITY;

ENTITY student SUBTYPE OF (person);
END_ENTITY;
```

Every person is either a child or an adult. A student is also either a child or an adult.

ONEOF

```
ENTITY person;
  first_name : STRING;
  last_name  : STRING;
  ss_no      : INTEGER;
  children   : SET [0:?] OF person;
UNIQUE
  un1 : ss_no;
END_ENTITY;

SUBTYPE_CONSTRAINT mf FOR person;
  ONEOF(male, female);
END_SUBTYPE_CONSTRAINT;

ENTITY male
  SUBTYPE OF (person);
  wife : OPTIONAL female;
END_ENTITY;

ENTITY female
  SUBTYPE OF (person);
  husband : OPTIONAL male;
END_ENTITY;
```

ANDOR

```
ENTITY person;
  first_name : STRING;
  last_name  : STRING;
  ss_no      : INTEGER;
  children   : SET [0:?] OF person;
UNIQUE
  un1 : ss_no;
END_ENTITY;

SUBTYPE_CONSTRAINT es FOR person;
  employee ANDOR student;
END_SUBTYPE_CONSTRAINT;

ENTITY employee
  SUBTYPE OF (person);
  salary : REAL;
END_ENTITY;

ENTITY student
  SUBTYPE OF (person);
```

```
  fees : REAL;
END_ENTITY;
```

AND

```
ENTITY person;
  ...
END_ENTITY;

SUBTYPE_CONSTRAINT mf_and_ca FOR person;
  ONEOF(male, female) AND
  ONEOF(citizen, alien);
END_SUBTYPE_CONSTRAINT;

ENTITY male SUBTYPE OF (person);
 ...
END_ENTITY;

ENTITY female SUBTYPE OF (person);
 ...
END_ENTITY;

ENTITY citizen SUBTYPE OF (person);
END_ENTITY;

ENTITY alien SUBTYPE OF (person);
END_ENTITY;
```

SUBTYPEs

- A Subtype is a specialisation of its Supertype(s).
- New attributes may be added.
- New constraints may be added.
- Attributes may be ‘retyped’ (i.e their domains may be specialised in a compatible manner).

SUBTYPEs (cont)

```
ENTITY circle;
  radius : NUMBER;
  center : point;
END_ENTITY;

ENTITY specialised_circle
  SUBTYPE OF (circle);
```

```
  SELF\circle.radius : REAL;    -- retyped
  shade  : colour; -- additional attribute
WHERE
  SELF\circle.radius > 3.0; -- add constraint
END_ENTITY;
```

QUERY Expression

The query expression evaluates a logical expression against each element of an aggregation, returning an aggregation of all the elements for which the logical expression is TRUE.

The syntax is roughly:

```
QUERY( temp <* agg | lexp)
```

where `temp` is the name of a temporary variable, `agg` is the aggregation, and `lexp` is the logical expression.

For example, assuming that a person's attributes included the age of the person,

```
QUERY(t <* persons | t.age >= 21)
```

would return all the people whose age was 21 or greater.

QUERY (cont)

The effect of QUERY is similar to the pseudo-function below.

```
FUNCTION q(agg  : AGGREGATE OF GENERIC;
           lexp : LOGICAL_EXPRESSION;)
          : AGGREGATE OF GENERIC;
LOCAL
  result : AGGREGATE OF GENERIC := [];
END_LOCAL;
  REPEAT i := 1 TO SIZEOF(agg);
    IF (lexp = TRUE) THEN
      result := result + agg[i];
    END_IF;
  END_REPEAT;
RETURN(result);
END_FUNCTION;
```

RULE

- Local constraints (WHERE, UNIQUE, INVERSE) are applied to each and every instance of the entity.
- Global constraints (RULEs) are applied between entities or across a subset of entity instances.

The following rule states that there shall be one and only one point at the origin in the objectbase.

```
RULE unique_origin FOR (point);
LOCAL
  origin : BAG OF point;
END_LOCAL;
  origin := QUERY(temp <* point |
                  (temp.x = 0.0) AND
                  (temp.y = 0.0) );
WHERE
  r1 : SIZEOF(origin) = 1;
END_RULE;
```

RULE (cont)

This RULE states that husbands and wives must be married to each other.

```
RULE married FOR (male,female);
  LOCAL
    ok1, ok2 : BOOLEAN := TRUE;
  END_LOCAL;
  IF (EXISTS(male.wife) AND
      male :<>: male.wife.husband) THEN
    ok1 := FALSE;
  END_IF;
  IF (EXISTS(female.husband) AND
      female :<>: female.husband.wife) THEN
    ok2 := FALSE;
  END_IF;
WHERE
  r1 : ok1;
  r2 : ok2;
END_RULE;
```

SCHEMA Interfacing

- Definitions within a Schema are potentially available to all Schemas.
- Definitions have to be ‘imported’ from the original Schema into the ‘current’ Schema.

- An imported definition implicitly imports all the necessary definitions to complete the definition.

EXPRESS syntax is roughly

```
import FROM schema_ref (def1 AS newname1,
                        def2 AS newname2);
```

USE Import

- Only ENTITYs and TYPEs can be imported via a USE statement.
- USEd ENTITYs are 'first class' items (i.e they can be independently instantiated).
- The 'stuff' required to complete the definitions of an imported item are implicitly REFERENCEd into the schema.
- If no list is given, *all* ENTITYs and TYPEs in the SCHEMA are imported.
- USEs can be chained.

NOTE

If `fc` is a first-class entity, then the statement

```
SIZEOF(USEROF(fc)) >= 0;
```

holds.

USE (cont)

```
SCHEMA source;
  ENTITY e1;
    attr : t1;
  END_ENTITY;

  TYPE t1 = REAL; END_TYPE;
END_SCHEMA;

SCHEMA using;
  USE FROM source (e1);

  ENTITY e2;
    attr : SET OF e1;
  END_ENTITY;
END_SCHEMA;
```

gives effectively:

USE (cont)

```
SCHEMA source;
  ENTITY e1;
    attr : t1;
  END_ENTITY;

  TYPE t1 = REAL; END_TYPE;
END_SCHEMA;

SCHEMA using;
  REFERENCE FROM source (t1);

  ENTITY e1;
    attr : t1;
  END_ENTITY;

  ENTITY e2;
    attr : SET OF e1;
  END_ENTITY;
END_SCHEMA;
```

REFERENCE Import

- Any kind of item can be imported via a REFERENCE statement.
- A REFERENCE is necessary to resolve references (links) to declarations in other schemas.
- REFERENCEDd items are 'second class' items (i.e they can not be independently instantiated).
- The 'stuff' required to complete the definitions of an imported entity are implicitly REFERENCEd into the schema.

NOTE

If sc is a second-class entity, then the statement

```
SIZEOF(USEROF(sc)) >= 1;
```

holds.

REFERENCE (cont)

```
SCHEMA source;
  ENTITY e1;
    attr : t1;
  END_ENTITY;
```

```
  TYPE t1 = REAL; END_TYPE;
END_SCHEMA;

SCHEMA referencing;
  REFERENCE FROM source (e1);

  ENTITY e2;
    attr : SET OF e1;
  END_ENTITY;
END_SCHEMA;
```

gives effectively:

REFERENCE (cont)

```
SCHEMA source;
  ENTITY e1;
    attr : t1;
  END_ENTITY;

  TYPE t1 = REAL; END_TYPE;
END_SCHEMA;

SCHEMA referencing;
  REFERENCE FROM source (e1, t1);

  ENTITY e2;
    attr : SET OF e1;
  END_ENTITY;
END_SCHEMA;
```

SCHEMA Interfacing

```
SCHEMA first;
  ENTITY aaa;
    -- attributes
  END_ENTITY;

  ENTITY original;
    attr : NUMBER;
  END_ENTITY;
END_SCHEMA; -- first

SCHEMA second;
  USE FROM first (aaa AS bbb);
  REFERENCE FROM first (original);
```

```
  ENTITY constrained
    SUBTYPE OF (original);
    attr : INTEGER(7);
    WHERE
      positive : attr > 0;
    END_ENTITY;
END_SCHEMA; -- second
```

EXPRESS Summary

- A powerful OO information modeling language
  - Primary form is a computer processible text language.
  - EXPRESS-G as a graphical subset.
  - EXPRESS-I as an instantiation form
  - EXPRESS-X transformation specification
- Is an ISO standard language.
- Normative STEP information models.
- Becoming widely used in the modeling communities.
- Software tools available.